Algoritmi e Strutture Dati

Visita di un albero (`visita-albero`)

Testo del problema

Dato un albero $T(V, E)$ radicato in $R \in V$, visitare l'albero in *pre-order*, *in-order* e *post-order*. Questo task ha 20 casi di test, di cui 6 sono alberi binari.

Visite in alberi arbitrari

Per attraversare alberi arbitrari (non necessariamente alberi binari) con una ricerca in profondità è possibile seguire il seguente algoritmo:

Algorithm 1 Attraversamento di un albero arbitrario

```
1:  function ATTRAVERSA(nodo)
2:      if nodo è vuoto then
3:          return
4:      end if
5:      VISITAPREORDINE(nodo)                                  ▷ pre-order
6:      for i ← 1 to numeroSottoalberi(nodo) − 1 do
7:          ATTRAVERSA(sottoalbero_i(nodo))
8:          VISITAINORDINE(nodo)                               ▷ in-order
9:      end for
10:     ATTRAVERSA(ultimoSottoalbero(nodo))
11:     VISITAPOSTORDINE(nodo)                                 ▷ post-order
12: end function
```

dove bisogna usare alternativamente VisitaPreOrdine, VisitaInOrdine, VisitaPostOrdine per la visita in *pre-order*, *in-order* e *post-order*.

Formato dell'input

L'input ha N righe: la prima riga contiene due interi, N e R, dove N è il numero di nodi ($|V|$) ed R è la radice dell'albero. Le successive $N - 1$ righe contengono due interi per riga $\{(u_i, v_i)\}_{i=1}^{N-1}$, il nodo di partenza e di arrivo degli archi. Gli archi sono specificati di modo che vadano sempre dai padri ai figli, questa condizione può essere espressa in questo modo:

$$\forall (u, v) \in E,\ u <_{\mathcal{T}} v \tag{1}$$

dove $<_{\mathcal{T}}$ è l'ordinamento topologico.

Formato dell'output

Tre righe: (1) nella prima riga, una visita dell'albero in *pre-order*; (2) nella seconda, una visita dell'albero in *in-order*; (3) nella terza, una visita dell'albero in *post-order*.

Assunzioni

- $1 \leq N \leq 50.000$
- $0 \leq R < N$
- $T(V, E)$ è un albero ed è connesso

Esempi di input/output

File `input.txt`	**File** `output.txt`
10 9 4 2 1 7 4 0 9 1 3 5 3 6 9 4 1 3 7 8	9 1 7 8 3 5 6 4 2 0 8 7 1 5 3 6 9 2 4 0 8 7 5 6 3 1 2 0 4 9

File `input.txt`	**File** `output.txt`
12 5 3 7 11 1 11 6 0 9 3 2 5 0 0 8 11 4 0 10 5 11 5 3	5 0 9 8 10 11 1 6 4 3 7 2 9 0 8 10 5 1 11 6 4 7 3 2 9 8 10 0 1 6 4 11 7 2 3 5

File `input.txt`	**File** `output.txt`
13 12 12 9 12 10 12 6 3 11 1 3 10 5 9 1 2 7 12 8 7 0 0 4 8 2	12 9 1 3 11 10 5 6 8 2 7 0 4 11 3 1 9 12 5 10 6 4 0 7 2 8 11 3 1 9 5 10 6 4 0 7 2 8 12